Beyond the Syllabus

Complexity Classes and Three 2024–2025 Breakthroughs in Algorithms

Week 2 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why look beyond the syllabus?

- This course teaches you **tools**: asymptotic analysis, greedy algorithms, divide and conquer, dynamic programming, network flow, NP-completeness.
- These tools are not museum pieces. Researchers are actively using them **right now** to settle questions that were open for 40–60 years.
- Today's supplement has two parts:
  1. A map of the **complexity classes** that organize "how hard is this problem, really?" – the language you need to read any theory paper.
  2. Three **2024–2025 breakthroughs** that each broke a decades-old speed barrier, using exactly the kind of reasoning ($T(n) = \ldots$, recurrences, worst-case analysis) you are learning this semester.

The message

"Textbook algorithm" does not mean "final answer." Dijkstra's algorithm is 70 years old and still generating Symposium-on-Foundations-of-Computer-Science best-paper awards.

What is a complexity class?

Definition (informal)

A **complexity class** is a set of computational problems that can be solved using a certain amount of a resource – usually **time** or **space** – as a function of input size n.

- You already know one: this whole course is implicitly about the class of problems solvable in **polynomial time**.
- Complexity theory asks a sharper question: *for a given problem, is polynomial time enough – and if we don't know, what can we prove about it anyway?*
- This matters practically: if a problem is provably "hard," you stop searching for a fast exact algorithm and instead look for approximations, heuristics, or special-case structure (topics in Weeks 12–15).

P: efficiently solvable problems

P (Polynomial time)

The class of decision problems solvable by an algorithm running in $O(n^k)$ time for some constant k, on a standard (deterministic) computer.

- Shortest paths, minimum spanning tree, sorting, matching, network flow – almost everything in Weeks 3–11 of this course lives in P.
- "Polynomial" is the standard mathematical stand-in for **"efficient."** $O(n^3)$ can be slow in practice, and $O(1.01^n)$ is technically not polynomial – but the P/not-P line is the most useful coarse-grained boundary theorists have found.

NP: efficiently *verifiable* problems

NP (Nondeterministic Polynomial time)

The class of decision problems where, *if the answer is "yes,"* there exists a proof (a **certificate**) that can be **checked** in polynomial time.

- Example: "Does this graph have a Hamiltonian cycle?" Nobody knows how to *find* one quickly, but if someone hands you a candidate cycle, you can *verify* it visits every vertex exactly once in $O(n)$ time.
- Every problem in P is also in NP (if you can solve it fast, you can certainly verify a solution fast): $\mathbf{P} \subseteq \mathbf{NP}$.
- The single biggest open question in computer science: **is** $P = NP$**?** Can everything that's easy to *check* also be made easy to *solve*?

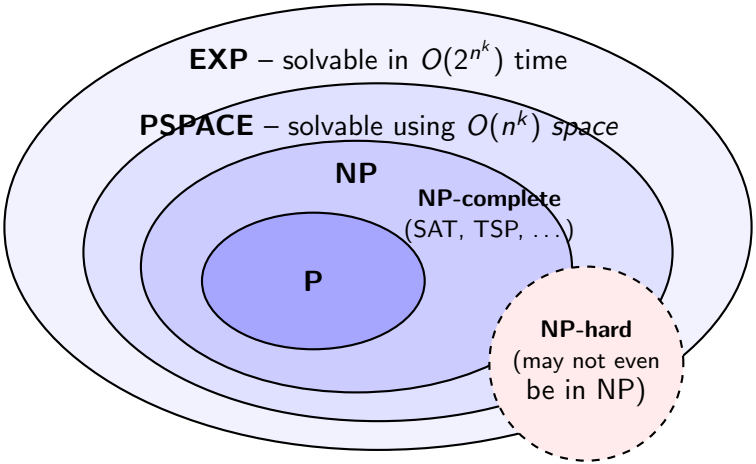
NP-complete and NP-hard

- A problem is **NP-hard** if every problem in NP can be reduced to it in polynomial time – informally, it is "at least as hard as anything in NP."
- A problem is **NP-complete** if it is NP-hard *and* itself in NP. NP-complete problems are the "hardest problems inside NP."
- Cook and Levin (1971) showed SAT (Boolean satisfiability) is NP-complete. Thousands of problems are now known to be NP-complete, including the **Traveling Salesman Problem** (decision version), 3-COLORING, and VERTEX COVER – you will meet several of these formally in Week 12.

Why this matters in practice

If you can reduce a known NP-complete problem to *your* problem, you have strong evidence your problem has **no** polynomial-time exact algorithm (unless $P = NP$). That is your cue to switch strategies: approximation (Week 14), randomization (Week 15), or exponential-but-practical search.

The big picture



- Known for sure: **P** $\subseteq$ **NP** $\subseteq$ **PSPACE** $\subseteq$ **EXP**.
- **Not** known: whether any of these inclusions is *strict*. (One is: $P \neq EXP$, by a classical diagonalization argument – but where the gap actually lives is wide open.)

P vs. NP: the $1,000,000 question

- *P* vs. *NP* is one of the seven **Clay Millennium Prize Problems** ($1,000,000 for a correct proof either way). Only one Millennium Problem has been solved so far (the Poincaré conjecture, by Grigori Perelman, 2003).
- Almost every theorist believes $P \neq NP$ – but belief is not proof, and the problem has resisted every attack for over 50 years.
- **Why you should care as a 2nd-year student:** every time you face a new problem in industry or research, "is this secretly NP-hard?" is one of the first questions worth asking, *before* you spend a week trying to write a fast exact algorithm.

Looking ahead

Week 12 makes all of this precise: formal reductions, Cook–Levin, and a toolkit for proving your own problems NP-hard.

Recap: Strassen's trick (1969)

- You saw last week that Karatsuba traded one multiplication for extra additions to beat the $\Theta(n^2)$ grade-school bound.
- Volker Strassen applied the *same philosophy* to **matrix multiplication**: multiplying two 2×2 block matrices needs only **7** multiplications (of the blocks), not the obvious **8**.
- Recursively applied to $n \times n$ matrices:

$$T(n) = 7\,T(n/2) + O(n^2) \implies T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807}),$$

beating the schoolbook $\Theta(n^3)$ – the first improvement in recorded history.

The exponent ω: a 50-year race

Define ω as the smallest exponent such that $n \times n$ matrices can be multiplied in $O(n^{\omega+\varepsilon})$ time for every $\varepsilon > 0$.

Year	Result	Bound on ω
1969	Strassen	$2.807\ldots$
1990	Coppersmith–Winograd	2.376
2014	Le Gall	2.3728639
2020–21	Alman–Williams	2.3728596
2022	Duan–Wu–Zhou	2.3719
2024	Williams–Xu–Xu–Zhou	2.371339

- Nobody knows the true value of ω; many suspect $\omega = 2$ is achievable, but every attempt to prove it has stalled.
- These are all **asymptotic** (large-n) results with enormous constants – not (yet) practical for the matrix sizes you'd use in a homework assignment.

A different question: the small-case puzzle

- Separately from the asymptotic race, there is a very concrete combinatorial question: **exactly how many scalar multiplications are needed to multiply two 4×4 matrices?**
- Applying Strassen recursively to a 4×4 matrix (as two levels of 2×2 blocks) uses $7 \times 7 = \mathbf{49}$ multiplications.
- For over 50 years, **49** was the best known scheme for 4×4 matrices – until 2022.

Why anyone cares about "49 vs. 47"

Small building blocks get used *recursively*. Shaving multiplications off the base case improves the constant in the asymptotic bound every level of recursion – exactly like the jump from 4 to 3 recursive calls did for Karatsuba.

AI-assisted discovery joins the search

Year	Who / what	Result for 4×4
1969–2021	Strassen (recursive)	49 multiplications
2022	DeepMind's **AlphaTensor**	47 multiplications (mod-2 arithmetic)
2024	Kauers & Moosbauer (JKU Linz)	48 multiplications (standard arithmetic)
2025	DeepMind's **AlphaEvolve**	48 multiplications (complex-valued)

- AlphaTensor (Nature, 2022) reframed matrix multiplication as a single-player game and used reinforcement learning to search for low-multiplication schemes – the first improvement on Strassen's 4x4 case in 55 years.
- AlphaEvolve (2025) is a large-language-model-driven *evolutionary code search*: it proposes, tests, and refines algorithm implementations directly.

The lesson

A problem being "settled since 1969" didn't mean it was actually settled – it meant nobody had looked hard enough, with the right tools.

Dijkstra's algorithm (1956) and the sorting barrier

- **Single-source shortest paths:** given a graph with n vertices, m edges, and a source s, find the shortest distance from s to every other vertex.
- Dijkstra's algorithm: repeatedly pick the *closest unvisited* vertex, "finalize" its distance, and relax its outgoing edges.
- In 1984, Fredman and Tarjan's **Fibonacci heap** made this run in $O(m + n \log n)$ time – matching a fundamental lower bound: any algorithm that finalizes vertices *in sorted order of distance* cannot beat the time it takes to **sort** n numbers.

The "sorting barrier"

For 40 years, this looked like the end of the road for general (real-weighted) graphs – *unless* you find a way to avoid sorting altogether.

2024: Dijkstra is "universally optimal"

- Haeupler, Rozhoň, Tětek, Hladík, and Tarjan (FOCS 2024 best paper) asked a different question: not "what is the worst case over *all* graphs," but **"is there one algorithm that is the best possible on every individual graph structure?"**
- They designed a new heap data structure and proved: a variant of Dijkstra's algorithm is **universally optimal** – for every fixed graph *shape*, no algorithm can beat it (for worst-case edge weights on that shape).
- A beautiful, 70-years-later vindication of Dijkstra's original 20-minute, pencil-free design.

2025: breaking the sorting barrier

- Just one year later, Duan, Mao, and collaborators (STOC 2025 best paper) proved the opposite-sounding result: an algorithm that **beats** Dijkstra's $O(m + n \log n)$ *worst-case* bound, on general directed, real-weighted graphs.
- Key idea: don't finalize vertices strictly by increasing distance.
  - Group nearby frontier vertices into **clusters**; explore one representative per cluster instead of scanning the whole frontier.
  - Use short, bounded bursts of the (usually slow) **Bellman–Ford** algorithm to find "influential" hub vertices worth exploring next.
- Because vertices are no longer finalized in sorted order, the sorting lower bound simply **does not apply** to this algorithm.

Reconciling the two results

2024 (Haeupler et al.)	**2025 (Duan, Mao et al.)**
"Best possible *per graph shape*," among algorithms that finalize vertices by increasing distance	"Faster *in the worst case over all graphs*," by abandoning distance-order finalization entirely

The teaching moment

Both statements are true. "Optimal" always means *optimal with respect to a model and a metric*. Changing the model (what the algorithm is allowed to do) or the metric (worst-case-over-graphs vs. per-graph-shape) can change the answer – a theme you will see again with approximation ratios in Week 14.

The problem: edge coloring

- **Motivating story:** at an airport, model runways/taxiways/gates as vertices and each aircraft's planned route as a path connecting them. Color each edge (route segment) so that no two edges sharing a vertex share a color – colors represent non-conflicting time slots.
- **Vizing's theorem** (1964): every graph can be edge-colored with at most $\Delta + 1$ colors, where Δ is the maximum vertex degree. Knowing *how many* colors suffice is easy.
- **Actually finding** such a coloring quickly is a different, much harder story.

A 40-year plateau

- Vizing's own algorithm runs in $O(mn)$ time (recoloring a chain of edges can cascade across the whole graph).
- 1980s improvements brought this down to $O(m\sqrt{n})$ – and then progress **stalled for roughly 40 years**.
- "The research stopped for a very long time ... many people believed that there's no better way," as one researcher put it.

2024–2025: near-linear time, finally

- 2024: two independent groups broke the 40-year plateau within days of each other: Assadi's $O(n^2)$-time algorithm, and a separate $O(m\sqrt[3]{n})$-time algorithm, later refined to $O(m\sqrt[4]{n})$.
- 2025: Assadi, Behnezhad, Costa, and collaborators combined forces and went much further – a **near-linear** $O(m \log m)$-time algorithm, presented at STOC 2025.
- Since simply *reading* the graph takes $\Omega(m)$ time, $O(m \log m)$ is within a single logarithmic factor of the **theoretical minimum possible**.

The key new idea: "priming"

Rather than trying to color the *last, hardest* edge faster (the old approach), use randomization to color *almost all* edges at once, leaving only a small, structured set of edges – which can then be colored quickly and nearly simultaneously.

Why these three stories matter

- Every one of these breakthroughs attacks a **classical, textbook** problem: matrix multiplication, shortest paths, edge coloring.
- Every one of these barriers stood for **decades** (40–55 years) and was widely believed to be permanent.
- Every one of these breakthroughs came from a genuinely **new idea** about combining old techniques (clustering, bounded Bellman–Ford bursts, randomized "priming"), not from more computing power alone – though AI-assisted search (AlphaTensor, AlphaEvolve) is now a serious new tool in the theorist's toolbox.

Summary

1. **Complexity classes** (P, NP, NP-complete, NP-hard, PSPACE, EXP) give you a vocabulary for asking "how hard, provably, is this problem?" – *P* vs. *NP* remains the field's central open question.
2. **Matrix multiplication:** the asymptotic exponent ω keeps inching down ($2.807 \to 2.371$), and AI-assisted search (AlphaTensor, AlphaEvolve) has now improved even the small, 55-year-old 4×4 case.
3. **Shortest paths:** Dijkstra's algorithm was proved *universally optimal* in 2024 – and its worst-case bound was still *beaten* in 2025, by an algorithm that refuses to sort. Both are correct; they optimize different things.
4. **Edge coloring:** a 40-year-stalled problem fell to near-linear time in 2024–2025 via a new "priming" idea.

Looking ahead

The tools behind every one of these results – recurrences, worst-case analysis, reductions, randomization – are exactly what the rest of this course builds up, week by week.